GenAI Project

Project Overview

- **Context**
  We are building an SMS-based chatbot that interacts with job candidates for a Python-developer position

- **Goal**
  Gather/verify information, answer questions, and ultimately schedule an interview with a human recruiter (or politely end the conversation)

- **Main Agent**
  Manages the dialogue turn-by-turn and decides among three actions:
  - Continue
  - Schedule
  - End

- **Advisor Agents** – Specialized helpers

  - Exit Advisor – Confirms when ending the conversation makes sense

  - Scheduling Advisor – Checks recruiter calendars (via SQL DB) & validates proposed slots

  - Info Advisor – Handles candidate questions related to the position, May utilize vector database information related to the position

Main Components

- **Main Agent**
  Orchestrates the conversation with the candidate. In each interaction, it must determine the appropriate course of action by selecting from the following options:

  - Option 1: Continue the Conversation
    If the candidate requests additional information or has further questions, the MAIN AGENT continues the dialogue by exchanging more messages

  - Option 2: End the Conversation
    If the candidate expresses disinterest (e.g., already found a job), the MAIN AGENT may decide to conclude the interaction

  - Option 3: Schedule an Interview
    If appropriate, the MAIN AGENT proceeds to schedule an interview, confirming date and time with the recruiter

- **Supporting Agents ("Advisors")**
  To manage complexity and maintain modularity, the MAIN AGENT consults specialized secondary agents (Advisors) at each decision point:

  - Conversation Exit Advisor
    Evaluates whether it is appropriate to end the conversation. This agent ensures that uninterested candidates are not unnecessarily followed up with, avoiding a negative experience

  - Interview Scheduling Advisor
    Assists in determining whether it's the right time to schedule an interview. It checks conversation context, validates proposed times, and may interact with external tools (e.g., recruiter calendars via function calling)

  - Conversation Info Advisor
    Helps answering candidate's questions, formulate the next message and maintain engagement. **Also aims to drive the conversation toward the end goal: scheduling an interview**

- **Evaluation Strategy**
  To assess the performance of the system, a labeled data set of real conversations is provided. Each response turn is annotated with the correct action the bot should take:
  - End the conversation
  - Schedule an interview
  - Continue the conversation

  These labeled examples should be used to evaluate model performance using metrics such as:
  - Accuracy
  - Confusion Matrix

Additional Implementation Steps

- **Fine-Tuning**
  Conversation Exit Advisor should be fine-tuned to detect conversation scenarios that are expected to conclude

- **Embedding**
  Separately from the main process, we will carry out an offline embedding step, converting the provided PDF into vector representations stored in a Chroma database

- **Agent Architecture**
  With LangChain & OpenAI API Endpoint and Models

- **Function Calling to interact with the SQL database**
  When the Scheduling Advisor retrieves available time slots, it analyzes the conversation context. For example, if the user mentions 'next Friday', it infers the current date from the time the conversation took place and combines it with the user's input. Based on this, it then suggests the three nearest available time slots

- **Streamlit**
  Instead of using SMS, the proof of concept (PoC) will be implemented in Streamlit and deployed to the Streamlit Community Cloud

- **Prompting Strategies**
  - Role's
  - API Parameter
  - Instructions Prompts
  - Few-Shot Learning

- **Evaluation & Testing**
  Use the labeled test cases to evaluate how well the multi-agent system performs on End/Continue/Schedule tasks

Project Structure

A Project must have:

- **Project Repository, that includes:**
  - Working with Git (and **.gitignore**)
  - Virtual Environment (**.venv** and **requirements.txt** file)
  - **.env** file (environment variables)
  - **README.md** file that includes project documentation, such as:
    - Project purpose
    - How to install and run the project locally
    - Basic usage examples
    - Project Structure
  - App (main application code)
    - **__init__.py** file (package initializer)
    - **main.py** file (the entry point of the application)
    - Modules
      Application modules, for: main app, Fine-Tuning, Embedding, etc
      - **__init__.py** file
      - **module.py** file (module implementation)
  - Streamlit
    - **__init__.py** file
    - **streamlit_main.py** file (Main Streamlit app)
  - Test
    - **test_evals.ipynb** - Detailed jupyter notebook with model performance metrics (Accuracy, Confusion Matrix)

- **Project Structure Overview**
  This is a suggested project structure ,it may vary and be adjusted as needed

```
├── .gitignore            # Specifies files to ignore in version control
├── README.md             # Project documentation
├── LICENSE               # License information
├── requirements.txt      # Python dependencies
├── .venv/                # Virtual environment (ignored by git)
├── .env                  # Environment variables (ignored by git)
├── app/                  # Main application code
│   ├── __init__.py       # Package initializer
│   ├── main.py           # Entry point of the application
│   └── modules/          # Application modules
│       ├── __init__.py   # Package initializer for modules
│       ├── module_1/     # Module 1
│       │   ├── __init__.py
│       │   └── m_1.py    # Module 1 implementation
│       └── module_2/     # Module 2
│           ├── __init__.py
│           └── m_2.py    # Module 2 implementation
├── streamlit_app/        # Streamlit UI code
│   ├── __init__.py
│   ├── streamlit_main.py      # Main Streamlit app
│   └── utils.py          # Helper functions for Streamlit components
└── tests/                # Test suite
    ├── test_main.py      # Tests for the main application
    └── test_evals.ipynb  # For testing performance metrics
```

Data & Resources

- **Workflow Diagram**
- **sms_conversations.json** - labeled data set of real conversations
- **Python Developer Job Description.pdf**
- **OpenAI account** – API keys for embeddings, chat, and fine-tuning
- **Chroma** – Local or cloud instance
- **db_Tech.sql** – Simple availability table seeded with demo slots
- **LangChain** – Agents, Memories, Tool

Important Note

The bot's process includes additional options and complexities that were not covered here, but had to be simplified for the scope of this project